

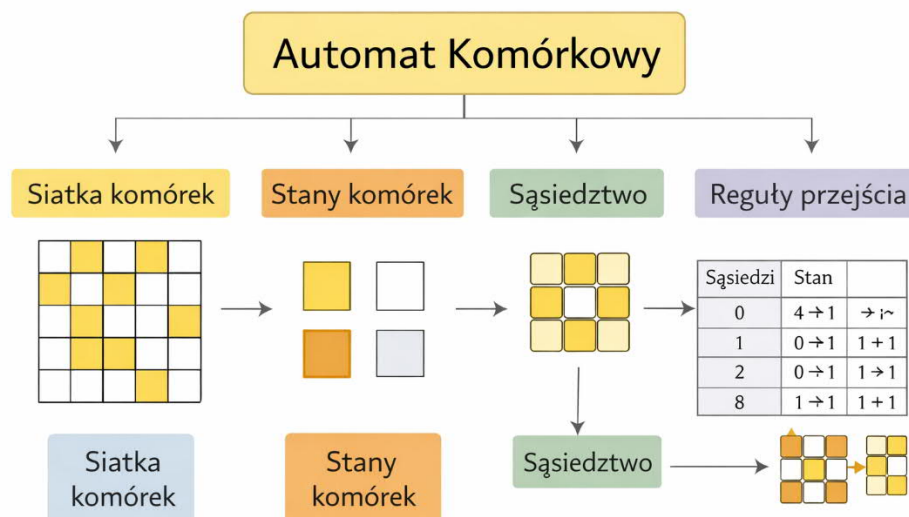
Automaty komórkowe

Definicja

Automat komórkowy (Cellular Automaton – CA) to model matematyczny służący do symulowania złożonych zjawisk przy użyciu prostych lokalnych reguł.

System składa się z siatki komórek, które zmieniają swój stan w kolejnych krokach czasu zgodnie z ustalonymi zasadami.

Podstawowe elementy automatu komórkowego



Automat komórkowy definiują cztery główne składniki:

1. Siatka komórek (grid)

- o jednowymiarowa, dwuwymiarowa lub wielowymiarowa
- o najczęściej stosowana jest siatka 2D

2. Stany komórek

Każda komórka może znajdować się w jednym z określonych stanów, np.:

- o 0 / 1
- o żywa / martwa
- o różne poziomy energii

3. Sąsiedztwo (neighborhood)

Określa, które komórki wpływają na zmianę stanu danej komórki.

Najczęściej stosowane:

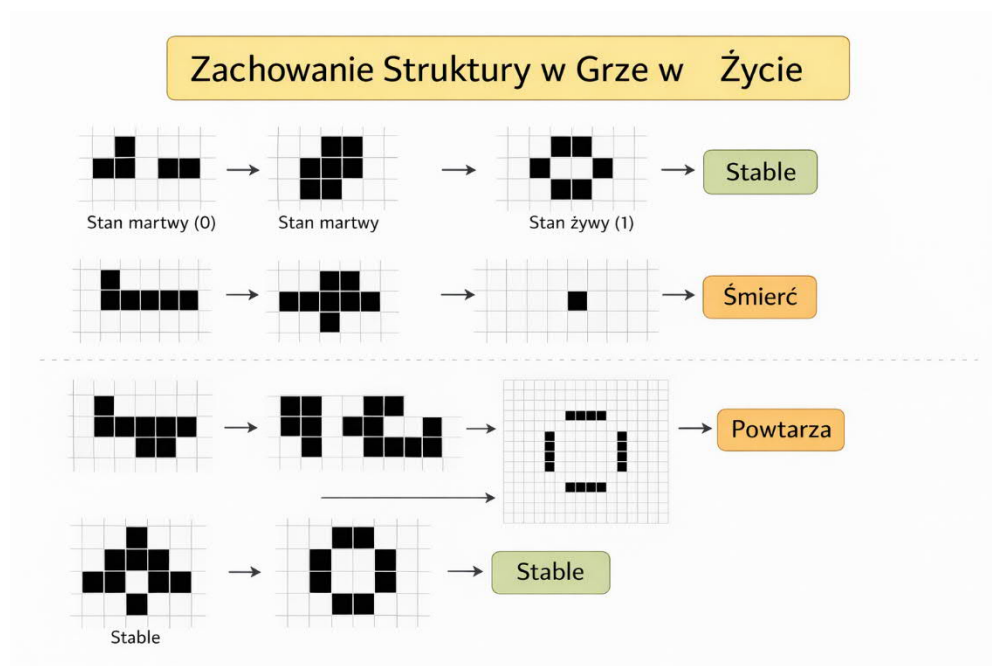
- o sąsiedztwo von Neumanna – 4 komórki
- o sąsiedztwo Moore'a – 8 komórek

4. Reguły przejścia (transition rules)

Określają, jak zmienia się stan komórki w następnym kroku czasu.

Stany komórek w automacie komórkowym – Gra w życie

W **Grze w życie (Game of Life)** każda komórka siatki może znajdować się tylko w **jednym z dwóch stanów**. Są to najprostsze możliwe stany w automacie komórkowym.



Stan martwy (0)

Komórka w tym stanie:

- jest **nieaktywna**,
- nie reprezentuje organizmu,
- może **pozostać martwa** lub **ożyć** w następnym kroku czasu.

Warunek narodzin:

- martwa komórka staje się żywa, gdy ma **dokładnie trzech żywych sąsiadów**.

Interpretacja biologiczna:

- pojawienie się nowego organizmu w populacji.

Stan żywy (1)

Komórka w tym stanie:

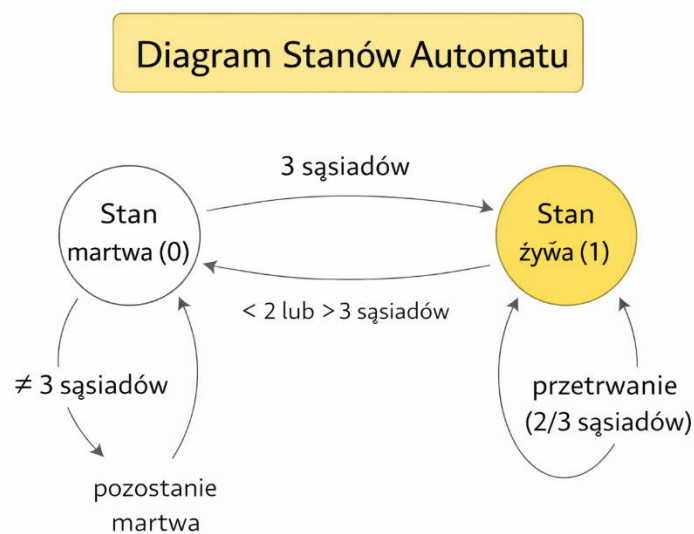
- reprezentuje **żywy organizm**,
- wpływa na stan komórek sąsiednich.

Może:

- **przeżyć**, gdy ma **2 lub 3 żywych sąsiadów**,
- **umrzeć z samotności**, gdy ma **mniej niż 2 sąsiadów**,
- **umrzeć z przeludnienia**, gdy ma **więcej niż 3 sąsiadów**.

Interpretacja biologiczna:

- dynamika populacji zależna od otoczenia.



Zmiana stanów komórki

<i>Stan obecny</i>	<i>Liczba żywych sąsiadów</i>	<i>Stan następny</i>
<i>martwa</i>	3	żywa
<i>martwa</i>	≠3	martwa

żywa	2 lub 3	żywa
żywa	<2 lub >3	martwa

Właściwości stanów w automacie

- system jest **dyskretny** (tylko dwa stany),
- zmiany zachodzą **jednocześnie dla wszystkich komórek**,
- nowy stan zależy tylko od **lokalnego sąsiedztwa**.

Podsumowanie

Zastosowania automatów komórkowych

Automaty komórkowe stosuje się w wielu dziedzinach:

Fizyka

- symulacje gazów
- dynamika płynów

Biologia

- wzrost populacji
- modele epidemii

Informatyka

- sztuczna inteligencja
- kryptografia
- modelowanie systemów złożonych

Inżynieria

- symulacja ruchu drogowego
- propagacja pożarów
- modelowanie materiałów

Zalety automatów komórkowych

- prostota reguł,
- możliwość modelowania złożonych systemów,
- łatwość implementacji komputerowej,
- dobra skalowalność

Wady

- duża wrażliwość na warunki początkowe,
- trudność analizy matematycznej,
- czasem duże zapotrzebowanie obliczeniowe

Automaty komórkowe są narzędziem umożliwiającym modelowanie **złożonych systemów przy użyciu prostych reguł lokalnych**.

Pomimo prostoty mogą generować bardzo skomplikowane i nieprzewidywalne zachowania.

Przykładowy kod gry w życie

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3. import matplotlib.animation as animation
4.
5. # --- konfiguracja ---
6. N = 80          # rozmiar siatki NxN
7. p = 0.2         # gęstość komórek żywych na start
8. INTERVAL_MS = 80 # szybkość animacji
9.
10. # losowy stan początkowy (0/1)
11. grid = (np.random.rand(N, N) < p).astype(np.uint8)
12.
13. def step(g):
14.     # liczba żywych sąsiadów (sąsiedztwo Moore'a, z zawijaniem na krawędziach)
15.     neigh = (
16.         np.roll(g, 1, 0) + np.roll(g, -1, 0) +
17.         np.roll(g, 1, 1) + np.roll(g, -1, 1) +
18.         np.roll(np.roll(g, 1, 0), 1, 1) +
19.         np.roll(np.roll(g, 1, 0), -1, 1) +
20.         np.roll(np.roll(g, -1, 0), 1, 1) +
21.         np.roll(np.roll(g, -1, 0), -1, 1)
22.     )
23.
24.     # reguły Conwaya:
25.     # - żywa komórka przeżywa przy 2 lub 3 sąsiadach
26.     # - martwa ożywa przy 3 sąsiadach
27.     survive = (g == 1) & ((neigh == 2) | (neigh == 3))
28.     born     = (g == 0) & (neigh == 3)
29.
30.     new_g = np.zeros_like(g)
31.     new_g[survive | born] = 1
32.     return new_g
33.
34. # --- wizualizacja ---
```

```
35. fig, ax = plt.subplots()
36. img = ax.imshow(grid, interpolation="nearest")
37. ax.set_title("Gra w życie (Conway)")
38. ax.set_axis_off()
39.
40. def update(_):
41.     global grid
42.     grid = step(grid)
43.     img.set_data(grid)
44.     return (img,)
45.
46. ani = animation.FuncAnimation(fig, update, interval=INTERVAL_MS, blit=True)
47. plt.show()
48.
```